МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ ____Старицин Олег Борисович_____
(подпись) (фамилия, имя, отчество)

Группы _____ИПо-41____

Руководитель практической подготовки от профильной организации

_____Специалист технической поддержки_____
(должность)

_____ _____Тюрин Андрей Дмитриевич_____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____Берестов Дмитрий Борисович_____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Оглавление

ВВЕДЕНИЕ 4

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 6

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 6

РАЗДЕЛ 2. Ознакомление со структурой и характером деятельности подразделения. 8

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении 8

Тема 2.2. Ознакомление с должностными и функциональными обязанностями 8

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 10

Тема 3.1. Выполнение производственных заданий 10

3.1.1. использование выбранной системы контроля версий; 10

3.1.2. разработка программного обеспечения; 10

3.1.2.1. Подробное описание программного кода 13

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества; 23

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования; 23

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств; 24

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; 25

3.1.7. Интегрирование программных модулей; 26

3.1.8. Верификация и аттестация программного обеспечения; 27

ЗАКЛЮЧЕНИЕ 28

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ..... 29

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

Выполнять интеграцию модулей в программное обеспечение;

Выполнять отладку программного модуля с использованием специализированных программных средств;

Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;

Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

Планировать и реализовывать собственное профессиональное и личностное развитие;

Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

Использовать информационные технологии в профессиональной деятельности

Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В первый день прохождения производственной практики был проведён инструктаж по правилам безопасного поведения на предприятии. Руководитель практики ознакомил с общими функциональными обязанностями сотрудника отдела разработки программного обеспечения, а также с правилами внутреннего распорядка организации.

Общие функциональные обязанности сотрудника: участие в разработке программных модулей согласно техническому заданию; написание и выполнение модульных тестов; документирование разработанного кода; соблюдение стандартов кодирования, принятых в организации; взаимодействие с командой разработчиков.

Правила техники безопасности на предприятии: запрещается приносить и употреблять алкогольные напитки; запрещается курить в помещениях офиса; необходимо соблюдать чистоту и порядок на рабочем месте; при обнаружении признаков пожара немедленно сообщать руководству и покидать здание согласно плану эвакуации.

Правила техники безопасности на рабочем месте программиста: рабочее место оборудовано персональным компьютером; расстояние от глаз до монитора — не менее 50 см; при длительной работе за компьютером рекомендуется делать перерывы каждые 45–60 минут; стул должен обеспечивать поддержку спины.

Правила безопасности при работе с электрическими приборами: запрещается самостоятельно вскрывать корпуса компьютерной техники; при обнаружении неисправности оборудования необходимо немедленно отключить его от сети и сообщить системному администратору; кабели

питания не должны располагаться в проходах; не допускается попадание жидкостей на электрооборудование.

РАЗДЕЛ 2. Ознакомление со структурой и характером деятельности подразделения.

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

Практика проходила в ООО «Специалист технической поддержки». Режим работы организации: понедельник–пятница с 9:00 до 18:00, обеденный перерыв с 13:00 до 14:00. Форма организации труда — офисная, с возможностью частичной удалённой работы.

Структура подразделения включает: отдел разработки программного обеспечения, отдел тестирования, отдел технической поддержки и администрацию. Непосредственное руководство осуществляет руководитель практики — Тюрин Андрей Дмитриевич, специалист технической поддержки.

Принцип управления — линейно-функциональный. Задачи ставятся руководителем через систему задач в системе контроля версий GitHub, где каждому разработчику назначаются конкретные задачи (Issues). Коммуникация ведётся по электронной почте и в мессенджере.

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

В рамках темы 2.2 была изучена должностная инструкция разработчика программного обеспечения. Основные права сотрудника: доступ к необходимым программным средствам и технической документации; получение задач в чётко определённой форме с указанием сроков выполнения; повышение профессиональной квалификации.

Основные обязанности: разработка программных модулей в соответствии с техническим заданием; участие в проектировании архитектуры программного обеспечения; написание технической

документации; проведение модульного тестирования разработанного кода; соблюдение стандартов кодирования (CamelCase для C#); участие в code review.

С руководителем практики было согласовано задание: разработка подсистемы для работы с партнёрами компании, включающей просмотр списка партнёров, добавление/редактирование данных о партнёре и просмотр истории реализации продукции. Цель практики — освоение профессиональных компетенций в области разработки и интеграции программных модулей.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Выполнение производственных заданий

3.1.1. использование выбранной системы контроля версий;

В качестве системы контроля версий была выбрана Git — распределённая система управления версиями, разработанная Линусом Торвальдсом в 2005 году. Git является наиболее распространённой системой контроля версий в современной разработке программного обеспечения.

Принцип работы Git основан на хранении снимков (snapshots) состояния файловой системы проекта. Каждый коммит (commit) фиксирует текущее состояние всех файлов проекта. Git позволяет создавать ветки (branch) для параллельной разработки различных функций, а затем объединять их (merge) в основную ветку.

Для удалённого хранения репозитория использовался сервис GitHub. Основные используемые команды Git: git init — инициализация репозитория; git add — добавление файлов в индекс; git commit — сохранение изменений; git push — отправка изменений на удалённый сервер; git pull — получение изменений с удалённого сервера; git branch — управление ветками; git merge — слияние веток. Исходный код проекта и сопроводительная документация размещены на GitHub по требованию руководителя практики.

3.1.2. разработка программного обеспечения;

Выбор модели разработки. Для данного проекта была выбрана итерационная модель разработки программного обеспечения. Итерационная модель предполагает разбиение процесса разработки на несколько итераций (циклов), каждая из которых включает этапы анализа, проектирования, реализации и тестирования. По завершении каждой итерации получается работающая версия программного продукта с определённым набором функций.

Обоснование выбора итерационной модели: задание содержит чётко определённые функциональные требования, допускающие поэтапную реализацию — сначала была создана структура базы данных и библиотека классов, затем основной WPF-интерфейс, затем дополнительный функционал управления продажами. Итерационный подход позволил своевременно выявлять ошибки и корректировать архитектурные решения на каждом этапе.

Инструменты, использованные для создания продукта: Microsoft Visual Studio 2022 — интегрированная среда разработки; язык программирования C# (.NET 8); технология WPF (Windows Presentation Foundation) для создания пользовательского интерфейса; Entity Framework Core 6 — ORM-фреймворк для работы с базой данных; СУБД PostgreSQL 15 для хранения данных; Git и GitHub для контроля версий; MSTest — фреймворк для написания модульных тестов; pgAdmin 4 — инструмент для администрирования PostgreSQL.

Среда программирования. Microsoft Visual Studio 2022 — это профессиональная интегрированная среда разработки (IDE) от компании Microsoft. Visual Studio 2022 поддерживает разработку приложений для Windows, веб-приложений, облачных сервисов. Среда предоставляет мощный отладчик, средства рефакторинга кода, встроенную поддержку Git, инструменты профилирования производительности и обширный набор расширений. Для данного проекта использовалась редакция Community (бесплатная для учебных целей).

Диаграмма классов представлена в Приложении к отчёту. Проект состоит из трёх сборок: StaritsinPartners.Domain — библиотека классов, содержащая модели (Partner, PartnerType, Product, PartnerSale), репозиторий (PartnerRepository), сервисы (DiscountService, SaleService), контекст БД (StaritsinDbContext) и фабрику контекста (StaritsinDbContextFactory); StaritsinPartners.WPF — WPF-приложение с окнами MainWindow,

PartnerFormWindow, SalesWindow, SaleEditWindow; StaritsinPartners.Tests — проект модульных тестов.

Этапы разработки ПО. На первом этапе было изучено техническое задание и спроектирована схема базы данных в третьей нормальной форме: таблицы partner_types, partners, products, partner_sales. Созданы необходимые первичные и внешние ключи, обеспечена ссылочная целостность. На втором этапе разработана библиотека классов StaritsinPartners.Domain, содержащая модели данных, репозиторий и сервисы бизнес-логики. На третьем этапе реализован WPF-интерфейс: главное окно со списком партнёров и историей продаж, форма добавления/редактирования партнёра, окно управления продажами. На четвёртом этапе написаны модульные тесты и проведено тестирование приложения.

Разработка форм для ввода, просмотра и редактирования данных. Реализованы следующие экранные формы: главное окно (MainWindow) с карточками партнёров и таблицей истории продаж; форма добавления/редактирования партнёра (PartnerFormWindow) с полями: наименование, тип, ФИО директора, телефон, email, адрес, рейтинг; окно управления продажами (SalesWindow) с таблицей всех продаж; форма добавления/редактирования продажи (SaleEditWindow) с автоматическим расчётом скидки и итоговой суммы.

Этапы разработки интерфейса. На этапе проектирования были изучены требования задания и определена структура окон приложения. На этапе прототипирования реализован базовый интерфейс с размещением элементов управления. На этапе стилизации в App.xaml определены единые стили для всего приложения: цветовая схема на основе зелёного цвета (#67BA80), стили кнопок (основная и вторичная), стили TextBox, ComboBox, DataGrid и меток полей форм. Все окна получили зелёную заголовочную полосу, единый шрифт и цветовую палитру.

3.1.2.1. Подробное описание программного кода

Архитектура проекта

Проект реализован по принципу многослойной архитектуры и состоит из трёх отдельных сборок: StaritsinLibrary — библиотека классов (.NET 8), содержащая всю бизнес-логику, модели данных, репозиторий, сервисы и контекст базы данных; StaritsinWPF — WPF-приложение (.NET 8), пользовательский интерфейс, не содержит прямых зависимостей от Entity Framework Core; StaritsinTests — проект модульных тестов (MSTest). Такое разделение обеспечивает независимость бизнес-логики от интерфейса.

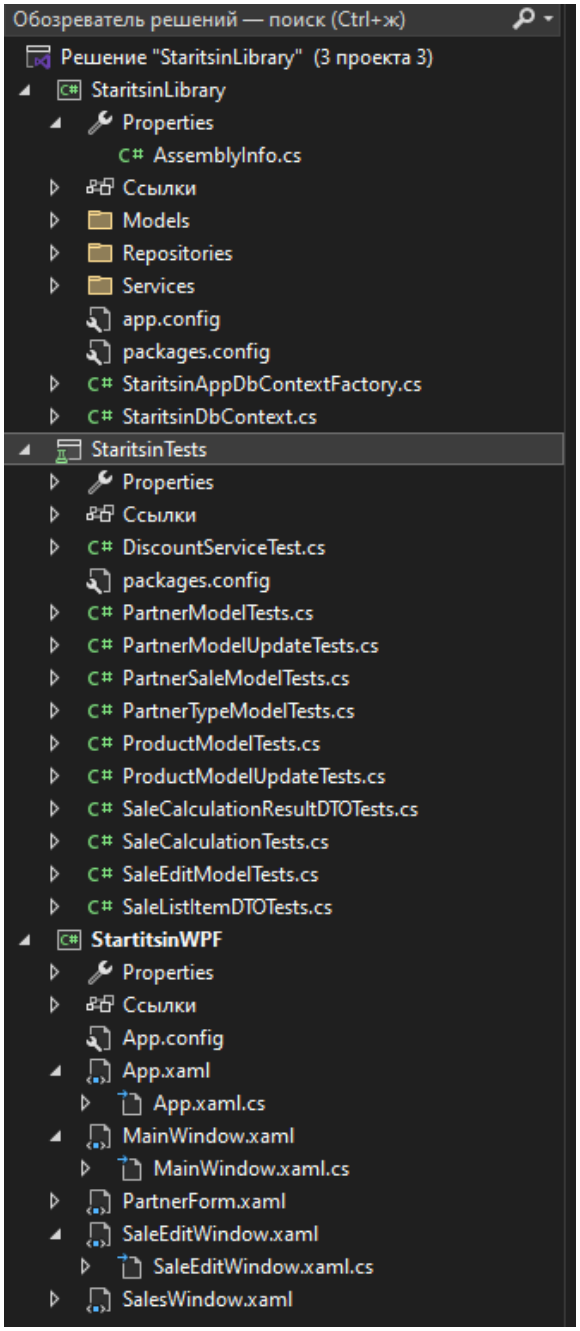

Рисунок 1. Структура решения StaritsinPartners в Visual Studio (Solution Explorer)

Модели данных (папка Models)

PartnerType.cs — соответствует таблице app.partner_types. Содержит Id (PK) и TypeName. Навигационное свойство Partners — связь «один ко многим». PartnerType.cs — соответствует таблице app.partners, хранит все данные партнёра. Атрибут [Range(0, int.MaxValue)] на поле Rating обеспечивает валидацию. Product.cs — таблица app.products, содержит

наименование, единицу измерения и базовую цену (decimal Price). PartnerSale.cs — таблица app.partner_sales, центральная таблица фактов. Поля BasePrice, DiscountPercent, UnitPrice сохраняются в момент продажи — это обеспечивает хранение исторической картины.

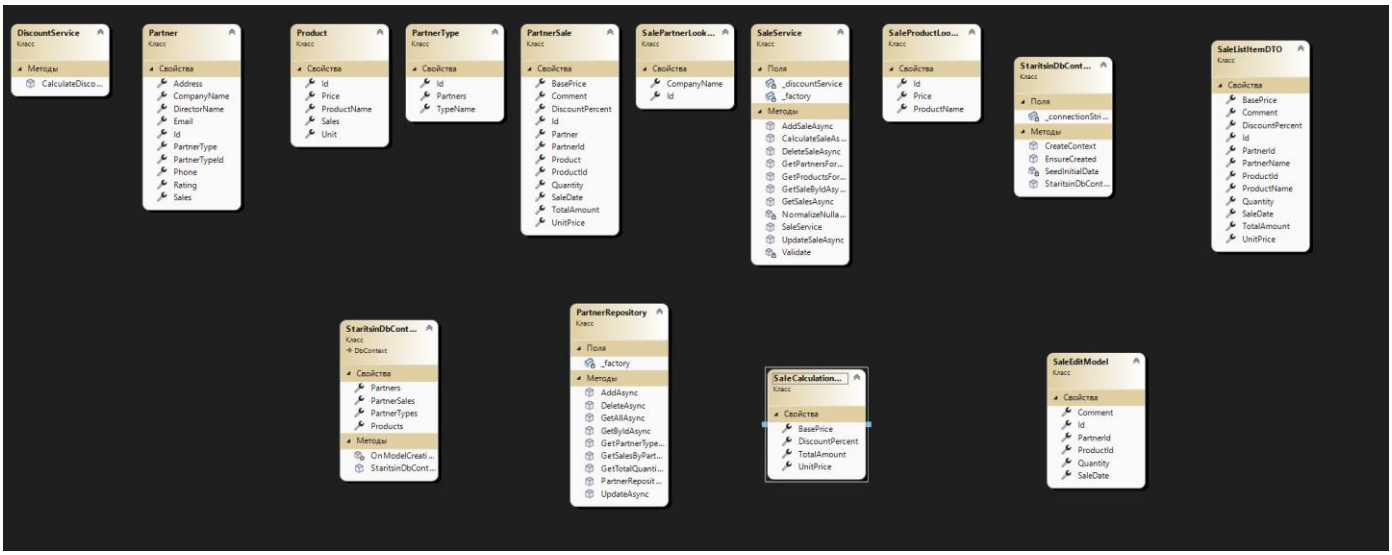

Рисунок 2. Диаграмма классов моделей (папка Models)

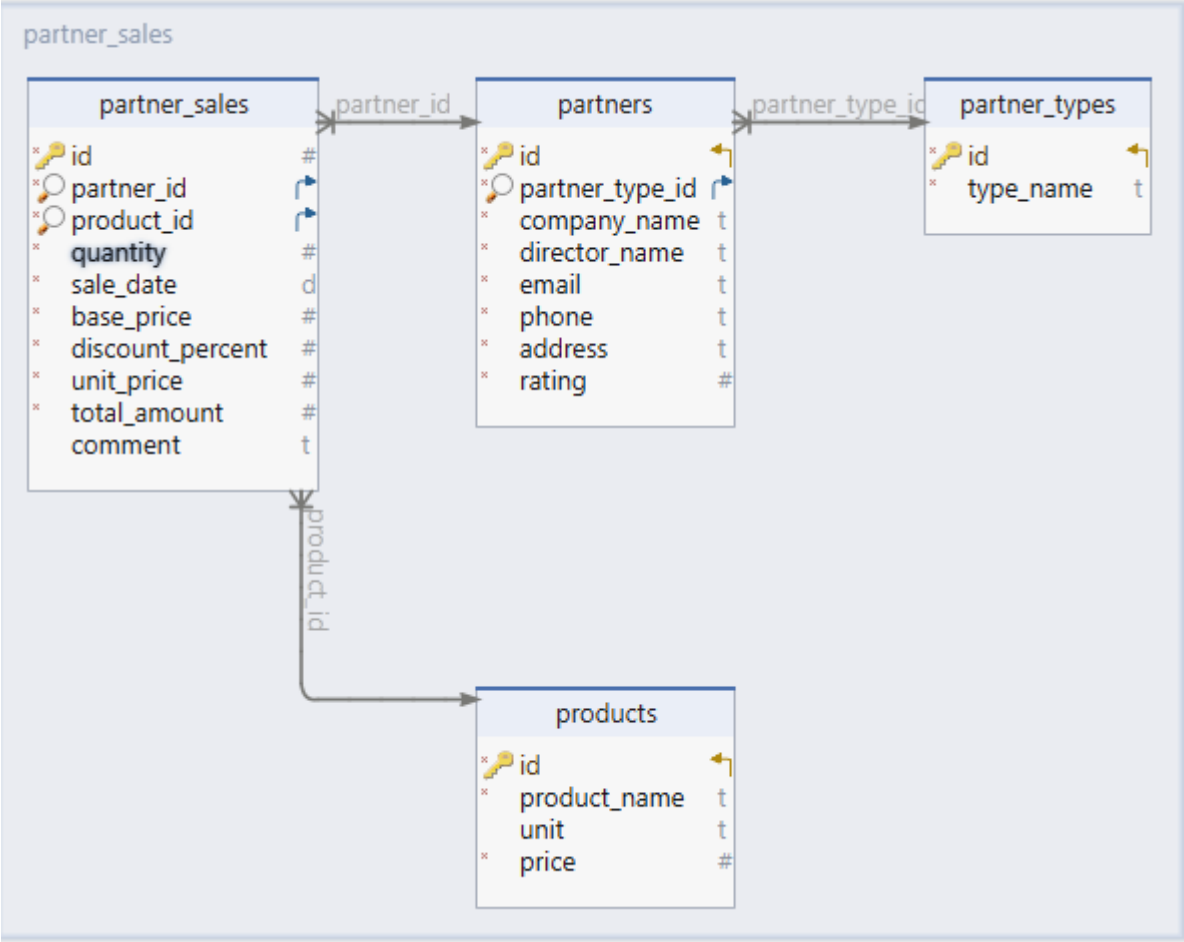

Рисунок 3. ER-диаграмма

StaritsinDbContext и StaritsinDbContextFactory

StaritsinDbContext наследует DbContext (Entity Framework Core) и содержит четыре свойства DbSet<T> — по одному на каждую таблицу. Метод OnModelCreating задаёт маппинг колонок (snake_case), типы данных (numeric(12,2) для денежных полей), каскадное удаление продаж при удалении партнёра и схему app по умолчанию.

```
public class StaritsinDbContext : DbContext
{
    Ссылок: 1
    public StaritsinDbContext(DbContextOptions<StaritsinDbContext> options) : base(options) { }

    Ссылок: 8
    public DbSet<Partner> Partners { get; set; }
    Ссылок: 2
    public DbSet<PartnerType> PartnerTypes { get; set; }
    Ссылок: 3
    public DbSet<Product> Products { get; set; }
    Ссылок: 10
    public DbSet<PartnerSale> PartnerSales { get; set; }

    Ссылок: 0
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema("app");

        modelBuilder.Entity<PartnerType>(entity =>
        {
            entity.ToTable("partner_types");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Id).HasColumnName("id");
            entity.Property(e => e.TypeName)
                  .HasColumnName("type_name")
                  .HasMaxLength(100)
                  .IsRequired();
        });

        modelBuilder.Entity<Partner>(entity =>
        {
            entity.ToTable("partners");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Id).HasColumnName("id");
            entity.Property(e => e.PartnerTypeId).HasColumnName("partner_type_id");
            entity.Property(e => e.CompanyName).HasColumnName("company_name").HasMaxLength(255).IsRequired();
            entity.Property(e => e.DirectorName).HasColumnName("director_name").HasMaxLength(255).IsRequired();
            entity.Property(e => e.Email).HasColumnName("email").HasMaxLength(255).IsRequired();
            entity.Property(e => e.Phone).HasColumnName("phone").HasMaxLength(50).IsRequired();
            entity.Property(e => e.Address).HasColumnName("address").HasMaxLength(500).IsRequired();
            entity.Property(e => e.Rating).HasColumnName("rating").HasDefaultValue(0);

            entity.HasOne(e => e.PartnerType)
                  .WithMany(t => t.Partners)
                  .HasForeignKey(e => e.PartnerTypeId);
        });

        modelBuilder.Entity<Product>(entity =>
        {
            entity.ToTable("products");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Id).HasColumnName("id");
            entity.Property(e => e.ProductName).HasColumnName("product_name").HasMaxLength(255).IsRequired();
            entity.Property(e => e.Unit).HasColumnName("unit").HasMaxLength(50).HasDefaultValue("шт");
            entity.Property(e => e.Price).HasColumnName("price").HasColumnType("numeric(12,2)").HasDefaultValue(0m);
        });

        modelBuilder.Entity<PartnerSale>(entity =>
        {
            entity.ToTable("partner_sales");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Id).HasColumnName("id");
            entity.Property(e => e.PartnerId).HasColumnName("partner_id");
            entity.Property(e => e.ProductId).HasColumnName("product_id");
            entity.Property(e => e.Quantity).HasColumnName("quantity");
            entity.Property(e => e.SaleDate).HasColumnName("sale_date");
            entity.Property(e => e.BasePrice).HasColumnName("base_price").HasColumnType("numeric(12,2)").HasDefaultValue(0m);
            entity.Property(e => e.DiscountPercent).HasColumnName("discount_percent").HasDefaultValue(0);
            entity.Property(e => e.UnitPrice).HasColumnName("unit_price").HasColumnType("numeric(12,2)").HasDefaultValue(0m);
            entity.Property(e => e.TotalAmount).HasColumnName("total_amount").HasColumnType("numeric(14,2)").HasDefaultValue(0m);
            entity.Property(e => e.Comment).HasColumnName("comment");

            entity.HasOne(e => e.Partner)
                  .WithMany(p => p.Sales)
                  .HasForeignKey(e => e.PartnerId)
```

Рисунок 4. Код DbContext

StaritsinDbContextFactory — фабрика контекстов. Решает ключевую архитектурную задачу: WPF-проект не содержит ссылок на EF Core. Метод EnsureCreated() при первом запуске выполняет три действия: создаёт схему app командой ExecuteSqlRaw (EF Core не умеет создавать PostgreSQL-схемы самостоятельно), затем вызывает Database.EnsureCreated() для создания таблиц, после чего заполняет их начальными данными через SeedInitialData().

```
Ссылок: 6
public class StaritsinDbContextFactory
{
    private readonly string _connectionString;

    Ссылок: 1
    public StaritsinDbContextFactory(string connectionString)
    {
        _connectionString = connectionString;
    }


    Ссылок: 17
    public StaritsinDbContext CreateContext()
    {
        var options = new DbContextOptionsBuilder<StaritsinDbContext>()
            .UseNpgsql(_connectionString)
            .Options;

        return new StaritsinDbContext(options);
    }

    Ссылок: 1
    public bool EnsureCreated()
    {
        using (var context = CreateContext())
        {
            context.Database.ExecuteSqlRaw("CREATE SCHEMA IF NOT EXISTS app");

            bool wasCreated = context.Database.EnsureCreated();

            if (wasCreated)
            {
                SeedInitialData(context);
            }

            return wasCreated;
        }
    }

    Ссылок: 1
    private static void SeedInitialData(StaritsinDbContext context)
    {
        var partnerTypes = new List<PartnerType>
        {
            new PartnerType { TypeName = "ЗАО" },
            new PartnerType { TypeName = "ООО" },
            new PartnerType { TypeName = "ПАО" },
            new PartnerType { TypeName = "ОАО" }
        };
        context.PartnerTypes.AddRange(partnerTypes);
        context.SaveChanges();

        var products = new List<Product>
        {
            new Product { ProductName = "Паркет тип 1",     Unit = "м²" },
            new Product { ProductName = "Паркет тип 2",     Unit = "м²" },
            new Product { ProductName = "Ламинат стандарт", Unit = "м²" },
            new Product { ProductName = "Ламинат премиум",  Unit = "м²" }
        };
        context.Products.AddRange(products);
        context.SaveChanges();

        var partners = new List<Partner>
        {
            new Partner
            {
                PartnerTypeId = partnerTypes[1].Id,
                CompanyName   = "СтройМастер",
                DirectorName  = "Иванов Иван Иванович",
                Email         = "info@stroymaster.ru",
                Phone         = "+7 495 123 45 67",
                Address       = "г. Москва, ул. Строителей, д. 1",
                Rating        = 7
            },
```

Рисунок 5. Код метода EnsureCreated() в StaritsinDbContextFactory.cs

PartnerRepository — репозиторий партнёров

Репозиторий инкапсулирует все операции с базой данных для сущности Partner. Каждый метод открывает контекст через using (var context = _factory.CreateContext()) — это предотвращает накопление соединений. Методы: GetAllAsync() — список всех партнёров с JOIN на PartnerType через Include(); GetByIdAsync(id) — один партнёр с типом и всеми продажами; AddAsync() / UpdateAsync() / DeleteAsync() — CRUD-операции; GetPartnerTypesAsync() — список типов для ComboBox; GetSalesByPartnerAsync(id) — история продаж партнёра; GetTotalQuantityByPartnerAsync(id) — суммарное количество через SumAsync(), защита от NULL через ?? 0.

```
Ссылок: 1
public async Task<List<Partner>> GetAllAsync()
{
    using (var context = _factory.CreateContext())
    {
        return await context.Partners
            .Include(p => p.PartnerType)
            .OrderBy(p => p.CompanyName)
            .ToListAsync();
    }
}
```

Рисунок 6. Код метода GetAllAsync() в PartnerRepository.cs

Сервисы: DiscountService и SaleService

DiscountService содержит единственный метод CalculateDiscount(int totalQuantity), реализующий алгоритм скидки через if/else: до 10 000 — 0%; 10 000–49 999 — 5%; 50 000–299 999 — 10%; от 300 000 — 15%. Метод не имеет зависимостей — поэтому его легко тестировать изолированно.

SaleService управляет продажами. Ключевой метод CalculateSaleAsync() получает суммарное количество продаж партнёра (исключая редактируемую запись), вычисляет скидку через DiscountService, рассчитывает UnitPrice = BasePrice * (1 - DiscountPercent / 100) с округлением до 2 знаков (MidpointRounding.AwayFromZero) и TotalAmount = UnitPrice * Quantity. Метод AddSaleAsync() вызывает CalculateSaleAsync() и сохраняет

рассчитанные цены вместе с записью — история продаж хранит фактические цены на момент продажи.

```
public async Task<SaleCalculationResultDTO> CalculateSaleAsync(
    int partnerId, int productId, int quantity, int? editingSaleId = null)
{
    if (partnerId <= 0)
        throw new ArgumentException("Не выбран партнёр.");

    if (productId <= 0)
        throw new ArgumentException("Не выбран продукт.");

    if (quantity <= 0)
        throw new ArgumentException("Количество должно быть больше нуля.");

    using (var context = _factory.CreateContext())
    {
        var product = await context.Products
            .AsNoTracking()
            .FirstOrDefaultAsync(x => x.Id == productId);

        if (product == null)
            throw new InvalidOperationException("Продукт не найден.");

        var totalQuantity = await context.PartnerSales
            .AsNoTracking()
            .Where(x => x.PartnerId == partnerId &&
                        (!editingSaleId.HasValue || x.Id != editingSaleId.Value))
            .SumAsync(x => (int?)x.Quantity) ?? 0;

        int discountPercent = _discountService.CalculateDiscount(totalQuantity);

        decimal unitPrice = Math.Round(product.Price * (1m - discountPercent / 100m), 2,
                                       MidpointRounding.AwayFromZero);
        decimal totalAmount = Math.Round(unitPrice * quantity, 2,
                                       MidpointRounding.AwayFromZero);

        return new SaleCalculationResultDTO
        {
            BasePrice = product.Price,
            DiscountPercent = discountPercent,
            UnitPrice = unitPrice,
            TotalAmount = totalAmount
        };
    }
}
```

Рисунок 7. Код метода CalculateSaleAsync() в SaleService.cs

WPF-приложение: App.xaml.cs и главное окно

App.xaml содержит глобальные стили: цветовая палитра (PrimaryBrush #67BA80), стили кнопок (PrimaryButtonStyle, SecondaryButtonStyle), стили TextBox, ComboBox, DataGrid. App.xaml.cs в методе OnStartup() создаёт объекты StaritsinDbContextFactory, PartnerRepository, DiscountService и SaleService, сохраняет их в статических свойствах App.PartnerRepository, App.DiscountService, App.SaleService — доступных из любого окна.

MainWindow разделён на две панели с GridSplitter. Левая панель — карточки партнёров (ItemsControl + DataTemplate) с поиском. Правая — DataGrid с историей продаж. Класс PartnerViewModel оборачивает модель Partner и добавляет DisplayTitle («Тип | Наименование») и DiscountText («10%»). Метод LoadPartnersAsync() загружает партнёров и для каждого вычисляет скидку. ApplySearchFilter() фильтрует список локально без запроса к БД через IndexOf с OrdinalIgnoreCase.

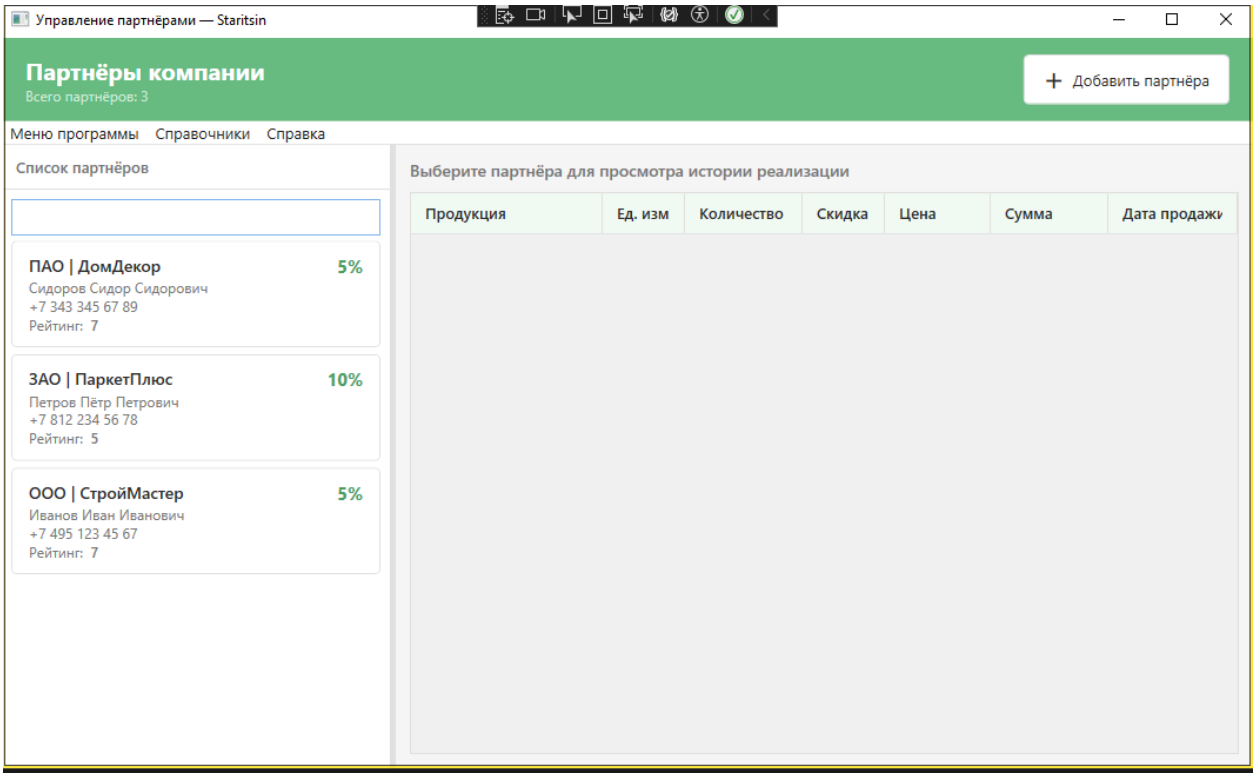

Рисунок 8. Главное окно приложения (MainWindow) с карточками партнёров и историей продаж

Формы ввода данных

PartnerFormWindow работает в двух режимах: editPartnerId = null — добавление (поля пусты), editPartnerId = int — редактирование (поля заполнены из БД). Валидация в методе ValidateForm() проверяет все обязательные поля, формат email (Regex), рейтинг >= 0. При ошибках показывается сводное сообщение. RatingBox_PreviewTextInput() разрешает только цифры — визуальная подсказка пользователю.

SaleEditWindow — форма продажи с автоматическим расчётом. Метод RefreshCalculationAsync() вызывается при каждом изменении полей партнёра,

продукта или количества и обновляет блок с базовой ценой, скидкой, ценой продажи и итоговой суммой. Флаг _isLoading предотвращает лишние пересчёты при первоначальном заполнении полей.

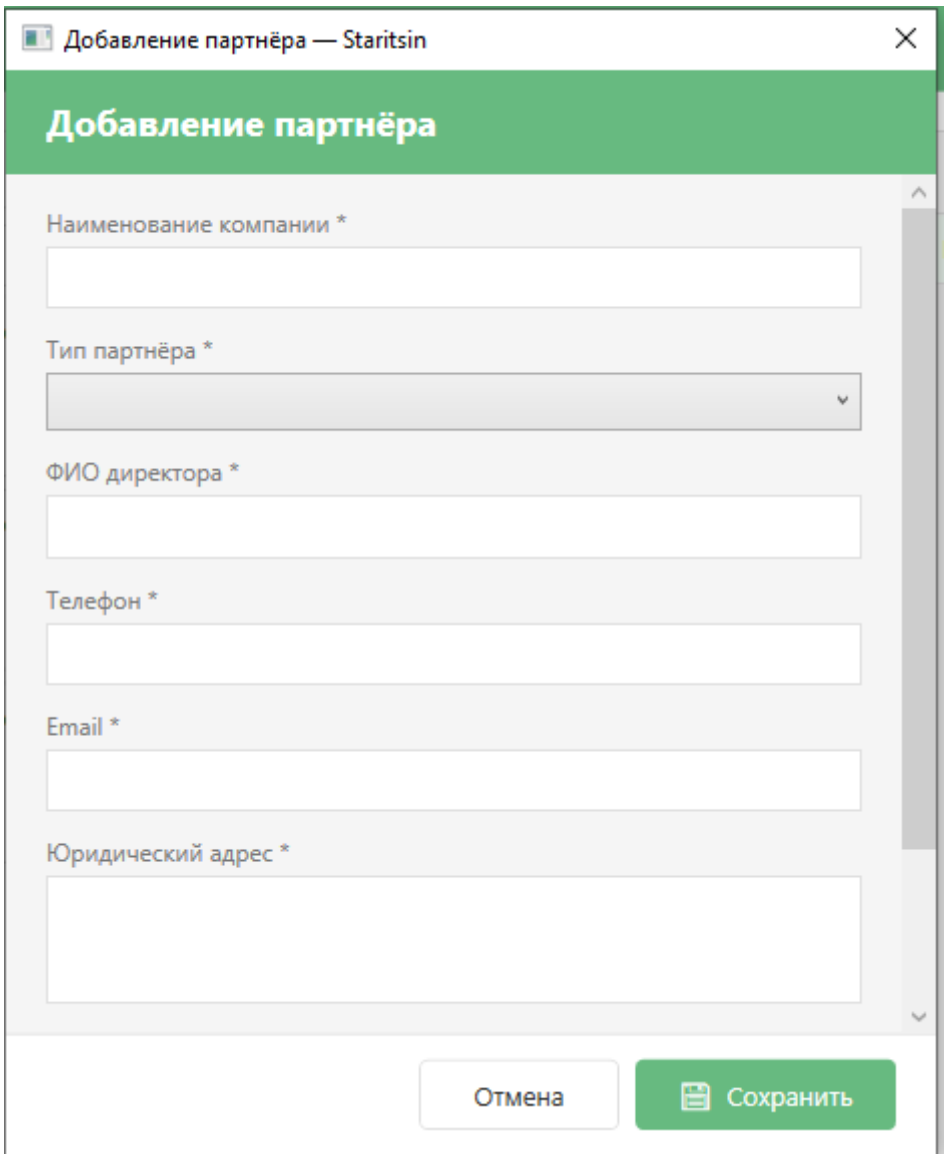

Рисунок 9. Форма добавления/редактирования партнёра (PartnerFormWindow)

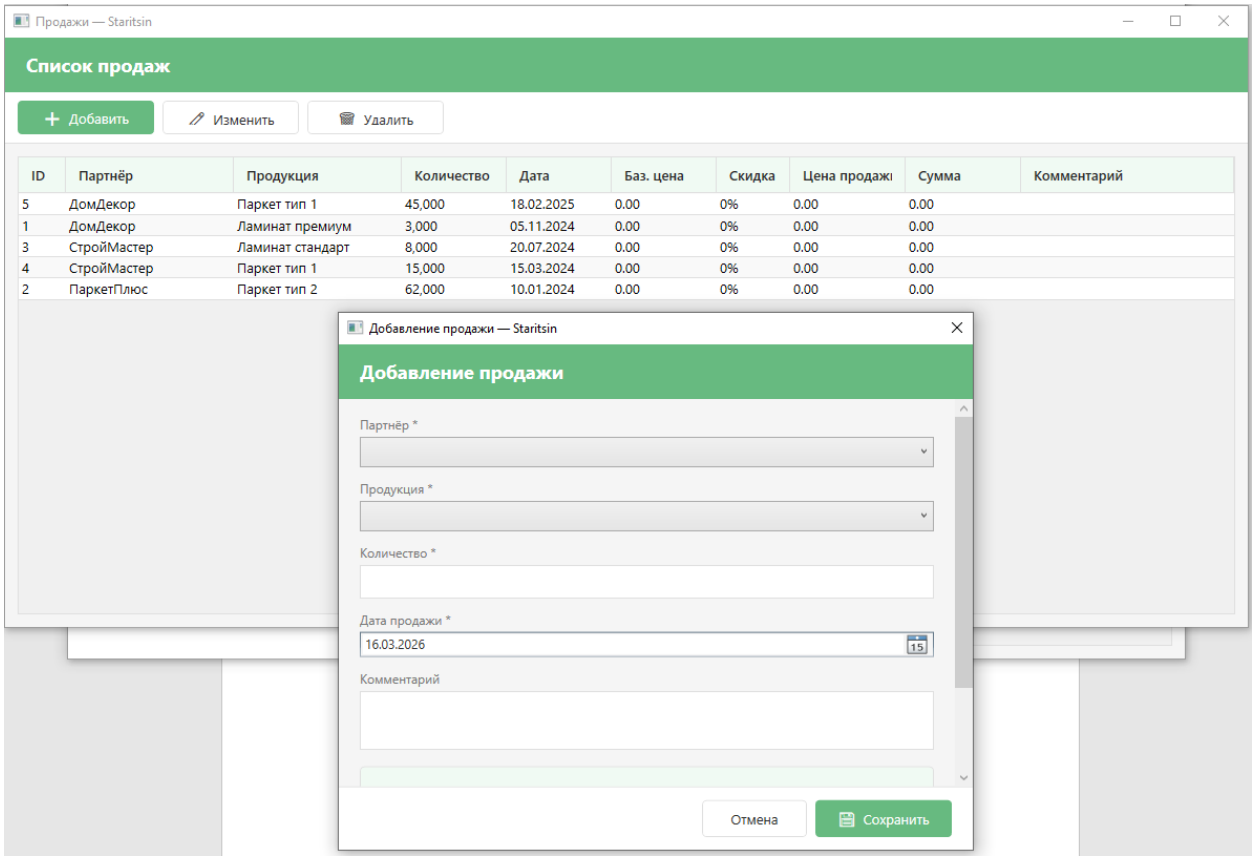

Рисунок 10. Форма добавления продажи (SaleEditWindow) с автоматическим расчётом скидки

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества;

В ходе разработки применялись следующие методы получения качественного кода. Паттерн Repository — для инкапсуляции логики доступа к базе данных в классе PartnerRepository, что обеспечивает разделение ответственности и упрощает тестирование. Паттерн Factory — класс StaritsinDbContextFactory отвечает за создание контекстов базы данных, не допуская прямого использования EF Core в WPF-слое.

Принцип единственной ответственности (SRP) — каждый класс выполняет одну задачу: DiscountService отвечает только за расчёт скидки, SaleService — за управление продажами, PartnerRepository — за CRUD-операции с партнёрами. Разделение на слои (Domain/WPF) обеспечивает независимость бизнес-логики от представления, что облегчает поддержку и масштабирование. Именование по стандарту CamelCase для всех идентификаторов, использование XML-документирования для публичных методов.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Виды инспекции кода. Статический анализ — проверка кода без его выполнения на соответствие стандартам и наличие потенциальных ошибок. Динамический анализ — проверка кода в процессе его выполнения (отладка, профилирование). Инспекция кода (Code Review) — ручной просмотр исходного кода коллегами с целью выявления ошибок и несоответствий стандартам.

Актуальные стандарты кодирования для C#. Microsoft C# Coding Conventions — официальные рекомендации Microsoft по стилю кода на C#, включающие правила именования (PascalCase для классов и методов, camelCase для локальных переменных), правила форматирования,

использования using-директив. StyleCop Analyzers — набор правил статического анализа для C#. В проекте применяется соглашение об именовании CamelCase согласно требованиям задания: все идентификаторы отражают своё назначение (PartnerRepository, DiscountService, CalculateDiscount).

Программы для инспекции кода. Microsoft Visual Studio 2022 — встроенный анализатор кода Roslyn выявляет ошибки и предупреждения в режиме реального времени. ReSharper (JetBrains) — расширение для Visual Studio, обеспечивающее расширенный анализ и рефакторинг. GitHub — позволяет проводить Code Review через Pull Requests. В ходе разработки использовался встроенный анализатор Visual Studio: все предупреждения анализатора были устранены.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

Отладка программного обеспечения проводилась с использованием встроенного отладчика Microsoft Visual Studio 2022. Процесс отладки включал следующие этапы: установка точек останова (breakpoints) в ключевых местах кода — в методах репозитория, сервисах и обработчиках событий WPF; пошаговое выполнение кода (Step Into, Step Over, Step Out) для отслеживания логики выполнения; просмотр значений переменных в окнах Watch, Locals и Autos.

В процессе отладки были выявлены и устранены следующие ошибки: ошибка подключения к PostgreSQL при отсутствии схемы app — устранена добавлением команды ExecuteSqlRaw(«CREATE SCHEMA IF NOT EXISTS app») перед вызовом EnsureCreated(); конфликт версий сборок System.Numerics.Vectors при использовании EF Core 8 с .NET Framework — устранён переходом на EF Core 6 и таргетинг net48; ошибка компилятора при использовании синтаксиса C# 8.0+ (switch expression, is not null) — код переписан для совместимости с C# 7.3.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения;

Тестирование программного обеспечения проводилось на двух уровнях: модульное тестирование (unit-тестирование) и функциональное тестирование. Модульные тесты были написаны с использованием фреймворка MSTest, входящего в состав Visual Studio. Тесты размещены в отдельном проекте StaritsinPartners.Tests.

Разработаны следующие тестовые классы: DiscountServiceTests — 10 тестов для всех граничных значений расчёта скидки (0%, 5%, 10%, 15%); PartnerModelTests — 10 тестов для проверки модели Partner; PartnerTypeModelTests — 4 теста; PartnerSaleModelUpdatedTests — 10 тестов для проверки полей модели PartnerSale; ProductModelUpdatedTests — 4 теста; SaleCalculationTests — 13 тестов расчёта цены и суммы продажи; SaleEditModelTests — 7 тестов; SaleListItemDTOTests — 7 тестов; SaleCalculationResultDTOTests — 7 тестов. Итого 72 модульных теста. Все тесты проходят успешно.

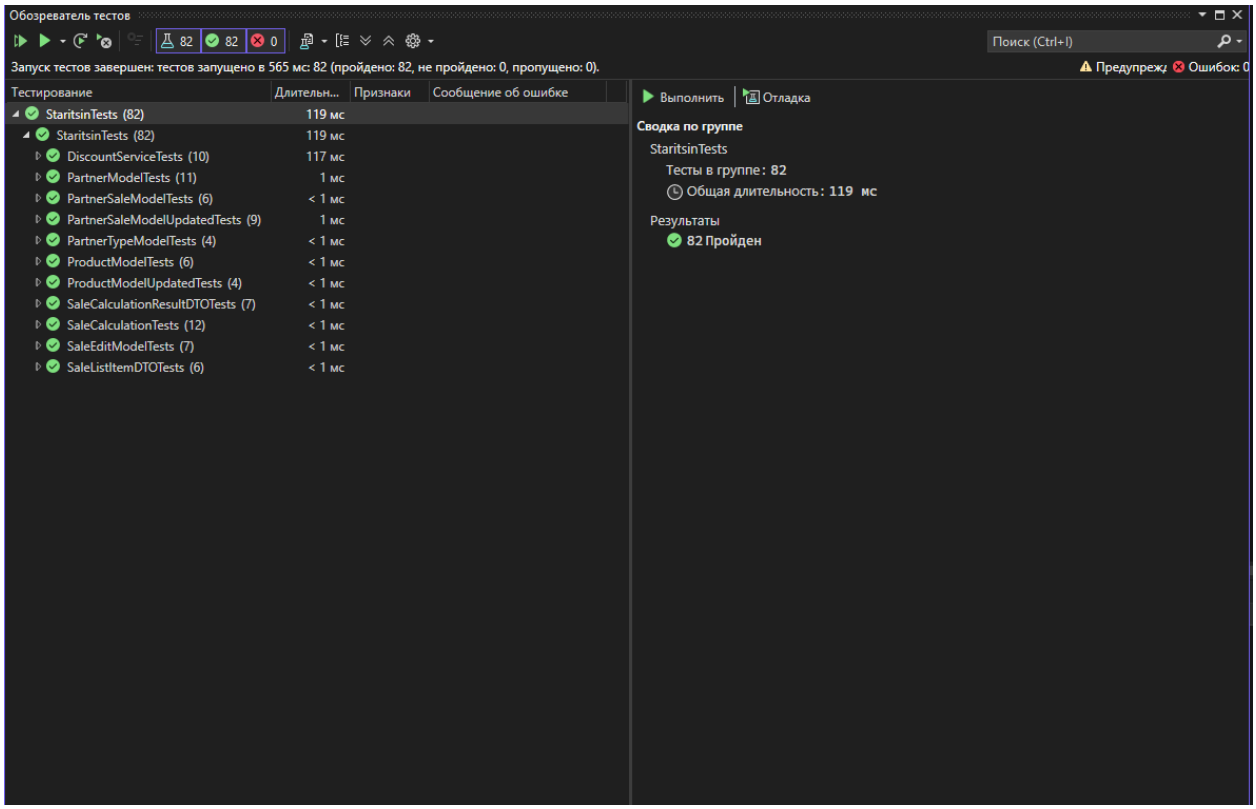

Рисунок 11. Модульные тесты в Visual Studio.

Функциональное тестирование включало: проверку отображения списка партнёров при запуске; проверку добавления нового партнёра с корректными и некорректными данными; проверку редактирования и удаления партнёра; проверку отображения истории продаж при выборе партнёра; проверку расчёта скидки при создании продажи; проверку валидации полей форм (рейтинг ≥ 0, корректный email и т.д.); проверку работы поиска по наименованию; проверку автоматического создания базы данных при первом запуске.

3.1.7. Интегрирование программных модулей;

Проект состоит из трёх программных модулей, объединённых в одно решение (solution) Visual Studio. Интеграция модулей осуществлена через механизм ссылок на проекты (Project References).

Модуль StaritsinPartners.Domain является центральным: он содержит все модели данных, контекст Entity Framework Core, фабрику контекста, репозиторий и сервисы. Модуль StaritsinPartners.WPF ссылается на Domain и использует его классы через пространство имён StaritsinPartners.Domain. В классе App.xaml.cs создаются экземпляры StaritsinDbContextFactory, PartnerRepository, DiscountService и SaleService, которые затем используются в окнах приложения через статические свойства App.PartnerRepository, App.DiscountService, App.SaleService. Модуль StaritsinPartners.Tests также ссылается на Domain для тестирования его сервисов и моделей.

Интеграция с базой данных PostgreSQL осуществляется через Entity Framework Core 6 и провайдер Npgsql. Строка подключения хранится в WPF-приложении и передаётся в StaritsinDbContextFactory, которая инкапсулирует все детали подключения внутри библиотеки классов. При первом запуске фабрика автоматически создаёт схему app и все необходимые таблицы через метод EnsureCreated(), после чего заполняет их начальными данными. Таким образом, WPF-модуль не содержит ни одной прямой ссылки на Entity

Framework Core или Npgsql — все взаимодействия с базой данных полностью инкапсулированы в Domain.

3.1.8. Верификация и аттестация программного обеспечения;

Верификация программного обеспечения — это процесс проверки того, что программный продукт соответствует своей спецификации и разработан правильно. В ходе верификации проверялось соответствие реализованного функционала требованиям технического задания.

Результаты верификации: требование «просмотр списка партнёров» — выполнено (главное окно отображает карточки партнёров с типом, наименованием, директором, телефоном, рейтингом и скидкой); требование «добавление/редактирование данных о партнёре» — выполнено (форма PartnerFormWindow содержит все обязательные поля, реализована валидация); требование «просмотр истории реализации» — выполнено (при выборе партнёра правая панель отображает таблицу продаж); требование «расчёт индивидуальной скидки» — выполнено (DiscountService корректно рассчитывает 0/5/10/15%); требование «3НФ с ссылочной целостностью» — выполнено (схема БД в 3НФ, внешние ключи настроены); именование CamelCase — соблюдено во всех классах и методах проекта.

Аттестация программного обеспечения — это процесс подтверждения того, что продукт соответствует потребностям пользователя и работает корректно в реальных условиях. В ходе аттестации было проведено комплексное тестирование приложения: запущено на рабочей машине под управлением Windows 10, проверена корректность всех операций CRUD для партнёров и продаж, проверена работа приложения при отсутствии базы данных (автоматическое создание), при некорректных данных (вывод понятных сообщений об ошибках). Приложение работает стабильно, аварийных завершений не наблюдалось. Все требования задания выполнены в полном объёме.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 17 марта 2026 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Разработана подсистема учёта партнёров компании на языке C# с использованием технологий WPF и Entity Framework Core, обеспечивающая полный функционал: просмотр списка партнёров, добавление/редактирование данных, просмотр истории реализации продукции и управление продажами.

2. Спроектирована и реализована база данных PostgreSQL в третьей нормальной форме с обеспечением ссылочной целостности, содержащая таблицы partner_types, partners, products, partner_sales. Реализовано автоматическое создание базы данных при первом запуске приложения.

3. Разработан алгоритм расчёта индивидуальной скидки партнёра на основе суммарного количества реализованной продукции (0%, 5%, 10%, 15%). Написано 72 модульных теста с использованием MSTest, покрывающих все граничные значения и основные сценарии.

4. Реализована архитектура приложения с чётким разделением на слои: библиотека классов StaritsinPartners.Domain (бизнес-логика и доступ к данным) и WPF-приложение StaritsinWPF (пользовательский интерфейс). WPF-модуль не содержит прямых зависимостей от Entity Framework Core.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Документация Microsoft по C# и .NET [Электронный ресурс]. — URL: https://docs.microsoft.com/ru-ru/dotnet/csharp/ (дата обращения: 16.03.2026).
2. Документация Microsoft по WPF [Электронный ресурс]. — URL: https://docs.microsoft.com/ru-ru/dotnet/desktop/wpf/ (дата обращения: 16.03.2026).
3. Документация Entity Framework Core [Электронный ресурс]. — URL: https://docs.microsoft.com/ru-ru/ef/core/ (дата обращения: 16.03.2026).
4. Документация PostgreSQL [Электронный ресурс]. — URL: https://www.postgresql.org/docs/ (дата обращения: 16.03.2026).
5. Метанит. Руководство по PostgreSQL [Электронный ресурс]. — URL: https://metanit.com/sql/postgresql/ (дата обращения: 16.03.2026).
6. Метанит. Руководство по C# [Электронный ресурс]. — URL: https://metanit.com/sharp/tutorial/ (дата обращения: 16.03.2026).
7. Метанит. Руководство по WPF [Электронный ресурс]. — URL: https://metanit.com/sharp/wpf/ (дата обращения: 16.03.2026).
8. Метанит. Руководство по Entity Framework Core [Электронный ресурс]. — URL: https://metanit.com/sharp/efcore/ (дата обращения: 16.03.2026).
9. Документация Microsoft по модульному тестированию в Visual Studio [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/visualstudio/test/walkthrough-creating-and-running-unit-tests-for-managed-code (дата обращения: 16.03.2026).